OnlineJudge

High Level Design Document

Stack: MongoDB • Express.js • React • Node.js

---

1. Overview

OnlineJudge is a full-stack competitive programming platform built with the MERN stack. Users can browse problems, write solutions in a browser-based coding environment, and receive automatic verdicts — similar to platforms like LeetCode or Codeforces.

The platform handles code submission securely by isolating execution inside Docker containers, ensuring that user-submitted code cannot affect the host system or consume unbounded resources.

2. Features

- User registration and login with JWT-based authentication
- Browse a list of problems with difficulty tags (Easy / Medium / Hard)
- Problem detail page with a full coding arena — code editor, language selector, run, submit, and test case panels
- Automatic solution evaluation against hidden test cases
- Verdict returned: Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error, or Compilation Error
- Global leaderboard ranking users by number of unique problems solved
- Practice problems that do not affect rankings

3. Supported Languages

For the initial version, the judge will support the following languages:

Language	Compiler / Runtime	File Extension
C++	g++	.cpp
Java	javac / JVM	.java
Python	Python 3	.py

4. Database Design (MongoDB)

The application uses four MongoDB collections:

4.1 users

Field	Type	Description
_id	ObjectId	Auto-generated primary key
fullName	String	Full name of the user
email	String (unique)	Login email address
passwordHash	String	bcrypt-hashed password
createdAt	Date	Account creation timestamp

4.2 problems

Field	Type	Description
_id	ObjectId	Auto-generated primary key
title	String	Problem name shown on listing
statement	String	Full problem description
difficulty	Enum	Easy | Medium | Hard
tags	String[]	Topic tags e.g. Arrays, DP
timeLimit	Number	Execution time limit in seconds
memoryLimit	Number	Memory limit in MB

4.3 testCases

Field	Type	Description
_id	ObjectId	Auto-generated primary key
problemId	ObjectId (ref)	Reference to problems collection

Field	Type	Description
input	String	Test case standard input
expectedOutput	String	Expected standard output
isSample	Boolean	Shown to user if true, hidden otherwise

4.4 submissions

Field	Type	Description
_id	ObjectId	Auto-generated primary key
userId	ObjectId (ref)	Reference to users collection
problemId	ObjectId (ref)	Reference to problems collection
code	String	Raw source code submitted
language	Enum	cpp | java | python
verdict	Enum	Accepted | Wrong Answer | TLE | CE | RE
submittedAt	Date	Auto-set submission timestamp

5. API Routes (Express.js)

5.1 Auth Routes — /api/auth

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive a JWT

5.2 Problem Routes — /api/problems

Method	Endpoint	Description
GET	/api/problems	Fetch all problems (list view)
GET	/api/problems/:id	Fetch a single problem + sample test cases

5.3 Submission Routes — /api/submissions

Method	Endpoint	Description
POST	/api/submissions	Submit code for evaluation (auth required)

Method	Endpoint	Description
GET	/api/submissions/leaderboard	Fetch top users ranked by problems solved

6. Frontend Screens (React)

Screen 1 — Home / Problem List

- Displays a table of all problems with: title, difficulty badge (colour-coded), and tags
- Each row links to the individual problem page
- Header has Login / Register buttons

Screen 2 — Problem / Coding Arena

- Left panel: problem statement, constraints, and sample test cases
- Right panel: code editor, language dropdown (C++ / Java / Python)
- Run button: executes against sample test cases and shows output inline
- Submit button: queues for full evaluation against all hidden test cases
- Verdict panel: shows result (Accepted, Wrong Answer, TLE, etc.) after evaluation completes
- Submissions tab (inside the statement panel): shows the user's past submission history for that problem with verdict, language, and timestamp

Screen 3 — Leaderboard

- Ranks users by number of problems solved (Accepted submissions only)
- Columns: Rank, Username, Problems Solved, Languages Used
- Ties broken by earliest date of last accepted submission

7. Evaluation System

7.1 Execution Flow

1. User submits code via the frontend
2. Backend receives the submission and places a job on a message queue (e.g. Bull + Redis)
3. A worker picks up the job asynchronously
4. Worker spins up a Docker container with the correct language image
5. Submitted code is written to a temp file inside the container
6. Container compiles (if needed) and runs the code against each test case input

7. Output is compared to expectedOutput for every test case
8. Verdict is saved to the submissions collection and returned to the frontend

7.2 Docker Sandboxing

- Each submission runs in an isolated container — no shared filesystem with the host
- Container is assigned a hard memory cap (e.g. 256 MB) to prevent memory abuse
- Execution is killed after the problem's timeLimit to enforce TLE
- Network access is disabled inside containers
- Containers are destroyed immediately after evaluation completes

8. Challenges & Solutions

Challenge	Solution
Thundering Herd — many simultaneous submissions	Message queue (Bull + Redis) buffers submissions; workers process them asynchronously
Malicious code (infinite loops, fork bombs, file access)	Docker container with memory cap, time limit, and no network access
Unauthorized manipulation of verdicts	JWT auth on all write routes; verdict written only by the worker, never by the client

9. Tech Stack Summary

Layer	Technology
Frontend	React
Backend	Node.js, Express.js
Database	MongoDB (via Mongoose)
Queue	Bull + Redis
Code Execution	Docker containers (g++, JVM, Python 3 images)
Auth	JWT (JSON Web Tokens) + bcrypt